

Reinforcement Learning

Planning and Learning with Tabular Methods: Part I

Min-hwan Oh

Seoul National University

Model-based vs. Model-free

So far, we learn model-based and model-free methods

- **Model-based** (e.g., Dynamic Programming) methods rely on **planning**
- **Model-free** (e.g., Monte Carlo, TD) methods rely on **learning**

Model-based vs. Model-free

So far, we learn model-based and model-free methods

- **Model-based** (e.g., Dynamic Programming) methods rely on **planning**
- **Model-free** (e.g., Monte Carlo, TD) methods rely on **learning**

These have similarities:

- Both revolve around the computation of value functions
- Both look ahead of future events, compute a backed up value and use it as an update target for an approximate value function

We will learn how model-free and model-based approaches can be combined.

Models

- A **model** is anything the agent can use to predict the environment's behaviour.
- Given a state and an action, a model predicts the next state and the next reward.
 - ▶ Output the whole probability distribution over the next states and rewards: **distribution models**
 - ▶ Produce only one of the possibility (the one with highest probability): **sample models**
- A model can be used to simulate the environment and produce simulated experience

Planning

- **Planning**: takes a model as input and produces or improves a policy for interacting with the modeled environment.

$$\text{model} \xrightarrow{\text{planning}} \text{policy}$$

- **State-space planning**: search through the state space for an optimal policy. Actions cause transitions from state to state, and value functions are computed over states
- **Plan-space planning**: search through the space of plans. Operators transform one plan into another (evolutionary methods, partial-order planning)

State-space planning

- Focus on state-space planning, that share a common structure:

model $\rightarrow$ simulated experience $\xrightarrow{\text{backups}}$ values $\rightarrow$ policy

- All state-space planning methods involve computing value functions as key intermediate towards improving the policy

State-space planning

- Focus on state-space planning, that share a common structure:

model $\rightarrow$ simulated experience $\xrightarrow{\text{backups}}$ values $\rightarrow$ policy

- All state-space planning methods involve computing value functions as key intermediate towards improving the policy
- Compute value functions by updates of backup operations applied to simulated experience.

State-space planning

- Focus on state-space planning, that share a common structure:

model $\rightarrow$ simulated experience $\xrightarrow{\text{backups}}$ values $\rightarrow$ policy

- All state-space planning methods involve computing value functions as key intermediate towards improving the policy
- Compute value functions by updates of backup operations applied to simulated experience.
- Dynamic Programming (DP) makes sweeps through the space of states, generating for each the distribution of possible transitions

Planning & Learning

- **Planning**: uses simulated experience generated by the model
- **Learning**: uses real experience generated by the environment
- In many cases, a learning algorithm can be substituted for the key update step of a planning method, e.g., Q-learning

Random-sample one-step tabular Q-planning

Loop forever:

1. Select a state, $S \in \mathcal{S}$, and an action, $A \in \mathcal{A}(S)$, at random
2. Send S, A to a sample model, and obtain
a sample next reward, R , and a sample next state, S'
3. Apply one-step tabular Q-learning to S, A, R, S' :
$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

Figure: Planning method based on Q-learning and random samples from a sample model

Dyna: Integrated Planning, Acting, and Learning

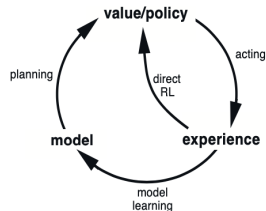
When planning is done online, while interacting with the environment, a number of issues arise.

- New information may change the model (and thus change planning)
- How to divide the computing resources between decision making and model learning?

Dyna-Q: simple architecture integrating the major functions of an online planning agent

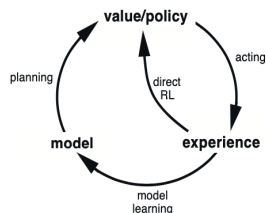
What do we do with real experience?

- **Model-learning**: improve the model (to make it more accurately match the environment)
- **Direct RL**: improve the value functions and policy used in the reinforcement learning programs we know
- **Indirect RL**: improve the value functions and policy via the model



What do we do with real experience?

- **Model-learning**: improve the model (to make it more accurately match the environment)
- **Direct RL**: improve the value functions and policy used in the reinforcement learning programs we know
- **Indirect RL**: improve the value functions and policy via the model



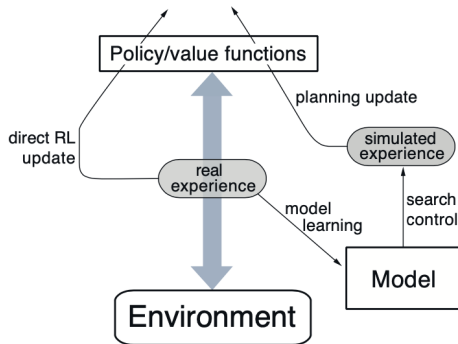
Both direct and indirect method have advantages and disadvantages:

- Indirect methods often make fuller use of limited amount of experience
- Direct methods are often simpler and not affected by bias in the design of the model

Dyna-Q includes all of the processes: planning, acting, direct RL and model-learning.

- Planning: random sample one-step tabular Q-learning
- Direct RL: method is one-step tabular Q-learning
- Model learning: assumes the environment is deterministic and
 - ▶ After each transition $S_t, A_t \rightarrow S_{t+1}, R_{t+1}$, the model records in its table entry for S_t, A_t the prediction that S_{t+1}, R_{t+1} will follow.
 - ▶ If the model is queried with a state-action pair it has seen before, it simply returns the last S_{t+1}, R_{t+1} experienced.
 - ▶ During planning, the Q-learning algorithm samples only from state-actions the model has seen

Architecture of Dyna-Q



- **Search control**: process that selects starting states and actions from the simulated experiences
- Planning is achieved by applying RL methods to simulated experience

Dyna-Q Pseudocode

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \epsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

- $Model(s, a)$: Predicted S_{t+1}, R_{t+1} for (s, a)
- Direct RL (d), model-learning (e), planning (f) steps
- If (e) and (f) were omitted?

Dyna-Q Pseudocode

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

- $Model(s, a)$: Predicted S_{t+1}, R_{t+1} for (s, a)
- Direct RL (d), model-learning (e), planning (f) steps
- If (e) and (f) were omitted? Then, the algorithm would be one-step (tabular) Q-learning

Dyna-Q Computational Challenge

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

- Conceptually, planning, acting, direct RL and model-learning (can) all occur in parallel. But for concreteness, we specify the order.

Dyna-Q Computational Challenge

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

- Conceptually, planning, acting, direct RL and model-learning (can) all occur in parallel. But for concreteness, we specify the order.
- Acting (b-c), direct RL (d) and model-learning (e) require little computation

Dyna-Q Computational Challenge

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

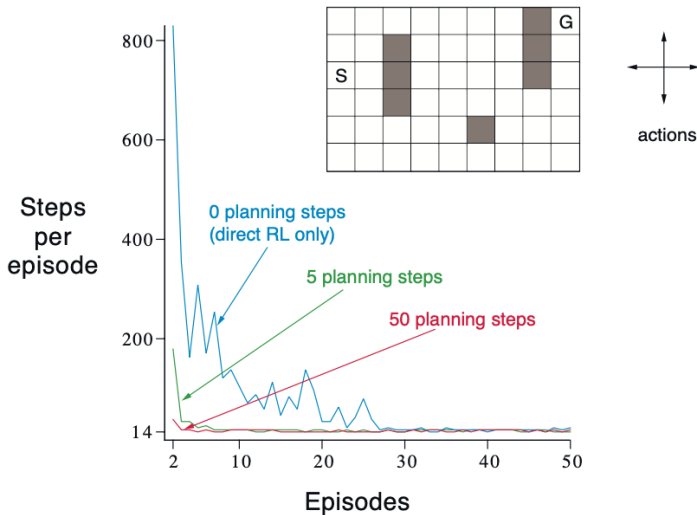
Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \epsilon\text{-greedy}(S, Q)$
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

- Conceptually, planning, acting, direct RL and model-learning (can) all occur in parallel. But for concreteness, we specify the order.
- Acting (b-c), direct RL (d) and model-learning (e) require little computation
- Planning (f) is inherently computationally intensive

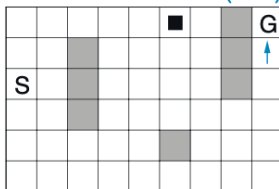
Example: Dyna Maze

- The task is to travel from **S** to **G** as quickly as possible

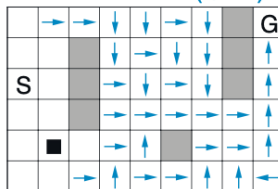


Non-planning vs. Planning: Dyna Maze

WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



- The black square is the agent
- The arrows denote the greedy action for the state (no arrow: all actions have equal value)
- These are the policies learned by a non-planning (left) and a planning (right) agent halfway through the second episode.

What is going on?

- Intermixing planning and acting: make them both proceed as fast as they can.
- Agent is always reactive, responding with the latest sensory information
- Planning is always done in the background, so is model-learning
- As new information is gained, the model is updated.
- As the model changes, the planning process will gradually compute a different way of behaving to match the newer model.

When the Model is Wrong

Models may be incorrect because:

- the environment is stochastic and only a limited amount of experience is available
- the model has learned using a function approximation that failed to generalize
- the environment has changed and new behaviour has not been observed yet

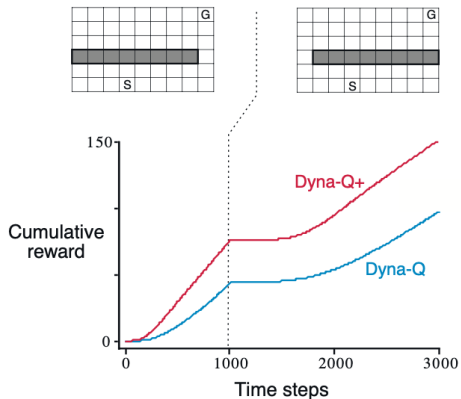
When the model is incorrect, the planning process is likely to compute a suboptimal policy

In some cases, following the suboptimal policy computed over a wrong model quickly leads to the discovery and correction of the modeling error.

- This can happen when the model is optimistic: predicting greater reward or better state transitions than are actually possible
- The planned policy attempts to exploit these opportunities.

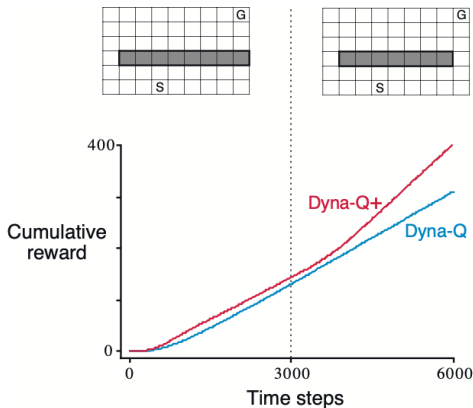
Non-stationary Environment

When the environment changes (the barrier shifts a little):



Non-stationary Environment

When the environment changes to offer a better alternative without changing the availability of the learned path:



Exploration-exploitation tradeoff:

- Want the agent to explore to find changes in the environment
- But not so much that the performance is greatly degraded.

Exploration-exploitation tradeoff:

- Want the agent to explore to find changes in the environment
- But not so much that the performance is greatly degraded.

Dyna-Q+: Simple heuristic approach

- Keeps track for each state-action pair of **how many time steps have elapsed** since last tried in a real interaction
- A special **bonus reward** is added to simulated experience involving these actions.
- If the transition has not been tried for over τ steps, then planning updates are done as if the transition produced a reward of $R + \kappa\sqrt{\tau}$ for some small κ .

Prioritized Sweeping

Can we do better than uniform sampling?

- For simulated transitions, state-action pair selected uniformly at random from all previously experience... Usually not the best.
- Focus on particular state-action pairs!
- Prioritize state-action pairs according to a measure of their urgency

Prioritized Sweeping

Can we do better than uniform sampling?

- For simulated transitions, state-action pair selected uniformly at random from all previously experience... Usually not the best.
- Focus on particular state-action pairs!
- Prioritize state-action pairs according to a measure of their urgency

Backward-focusing of planning computations: in general, we want to work back not just from goal states but from any state whose value have changed.

Prioritized Sweeping

Can we do better than uniform sampling?

- For simulated transitions, state-action pair selected uniformly at random from all previously experience... Usually not the best.
- Focus on particular state-action pairs!
- Prioritize state-action pairs according to a measure of their urgency

Backward-focusing of planning computations: in general, we want to work back not just from goal states but from any state whose value have changed.

Prioritized sweeping: A queue is maintained for every state-action pair whose value would change if updated, prioritized by the size of the change.

Next class: (Continue) Planning and Learning with Tabular Methods
(Chapter 8 of Sutton & Barto)